Maintenance Manual

Project name: **SX-Con**

Team name: **Stinger Systems**

Team Members

Alexander Bubienko

Colin Heinselman

Colin Henderson

Maksym Komarov

Joe Lee

Tim Liu

Tyler Slagboom

Kyle Valdez

Date: 5/2/2026

Table of Contents

1. Introduction

Project Logo

FIGURE 1.1: SX-CON LOGO

The project logo has dimensions of 351 by 55 pixels. It is embedded into the program via the paths:

- Main Window: src/imgs/sxc_window_logo.png
- Login Screen: src/imgs/sxc_logo.png

Changing the image files at the two paths will also change the program's embedded logo.

Description of the application

The application is a consignment software for a Southeast Asian grocery store called Super X Market. It is meant to take over their consignment method which is currently done with pen and paper. This enables keeping the data in the cloud to be able to have a non-physical copy of the consignment. The key features of the application are an easy-to-use interface and separate user and admin tools for creating and managing tickets.

Implementation tools and technologies

Programing languages Used, libraries, frameworks SDK's

- IDE:
  - Pycharm is recommended and used by the development team, though any Python IDE should work
- Programming languages:
  - Python 3.11.9
- Technologies, libraries and frameworks
  - pyqt6 (6.10.0): For building desktop applications with Qt.
  - azure-cosmos (4.14.2): For interacting with Azure Cosmos NoSQL database APIs.
  - bcrypt (5.0.0): For password hashing and security.

- openpyxl (3.1.5): For handling Excel files.
- pandas (3.0.0): For data manipulation and analysis, especially with Excel files.
- pywin32 (310): For Windows-specific functionalities, including printing.
- reportlab (4.4.0): For generating PDFs.

Database

- Azure Cosmos NoSQL

  Additional information can be found in section 5.

2. Troubleshooting

How to configure a computer for development work

The pyproject.toml file is included in the repository to enable quick and easy developer environment setup. Follow the steps provided in the next section to setup the developer environment.

What needs to be installed?

1. Install Python 3.11.9

    a. The following link leads to Python's official release of version 3.11.9

        https://www.python.org/downloads/release/python-3119/

    b. Use the appropriate link for your host

Windows 32bit: https://www.python.org/ftp/python/3.11.9/python-3.11.9.exe

Windows 64bit: https://www.python.org/ftp/python/3.11.9/python-3.11.9-amd64.exe

2. Install Pycharm

    While the development team used the professional version of Pycharm, the free community tier is more than capable of making changes to this repository.

    a. The following link leads to JetBrain's official release of PyCharm

        https://www.jetbrains.com/pycharm/download/?section=windows

3. Install Poetry v2.2.1

    a. Navigate to the terminal / command line of Pycharm

    b. Execute the following command:

        pip install poetry==2.2.1

4. Install the development environment

    a. Execute the following command to install the program dependencies:

        poetry install

How to get to the log files and examine them for errors

Log files are generated by the program autonomously and done so daily, assuming it is used daily. The logs can be found at the local reference of ./utils/logger/logs. These logs should not be pushed to the remote repository and should only be localized.

How to run automated tests

How to setup the test environment

The local test environment is self-contained and can run with the Pytest framework.

How to install Pytest

1. Navigate to the terminal / command line of Pycharm
2. Execute the following command:

    pip install pytest==9.0.2

3. Along with the environment setup mentioned in section 2: Troubleshooting, the test environment is now setup

How to run the test suite locally

With the assistance of pytest, the test suite can be run with just one command.

1. Navigate to the terminal / command line of Pycharm
2. Execute the following command:

    pytest -s -v

    the inclusion of the options “-s” and “-v” are to display verbose data should a test fail, this assists in debugging the issues

How to run the automated test suite using github action

With the assistance of github actions, the test suite can be run on the remote repository.

0. Log into github
1. Navigate to the github repository with the following link:

    https://github.com/ifndefy/SX-Con

2. Navigate to the “Actions” tab of the tab bar, alternatively use the following link:

    https://github.com/ifndefy/SX-Con/actions

3. In the left menu bar, select “Test PR”, alternatively use the following link:

    https://github.com/ifndefy/SX-Con/actions/workflows/PR_test.yaml

4. Select the “Run workflow” drop down menu
5. Select the dropdown menu from “Use workflow from”
6. From the dropdown menu, select the target branch to run the test suite on
7. Select “Run workflow” to begin the remote test suite

How to do “sanity” check to ensure code changes didn’t break anything

To ensure that changes do not break the program, the test suite should be run against the branch containing the changes prior to merging. Should all tests pass, it can be assumed that the program did not break the changes.

3. Deployment

Deployment can be done in two ways, executable or repository build.

The executable provides an easier setup that does not require an environment setup other than the config.ini which contains confidential information to connect the program to the database.

The repository build provides a faster program but requires an environment setup as well as the config.ini file.

Deployment via Executable

1. Navigate to the github repository via the following link:

https://github.com/ifndefy/SX-Con

2. Navigate to the “Releases” section or via the following link:

https://github.com/ifndefy/SX-Con/releases

3. Locate the latest release

4. Download the appropriate “Source code” zip to your local host

Windows: Source code (zip)

5. Extract the downloaded zip file

6. Create the config.ini file as guided in “How to setup the database config.ini”

6. Run the executable

Deployment via Repository

1. Navigate to the github repository via the following link:

https://github.com/ifndefy/SX-Con

2. Click on the “Code” drop down menu

3. Select “Download ZIP”

4. Extract the downloaded zip file

5. Create the config.ini file as guided in “How to setup the database config.ini”

5. Run the python file “SXC.py” to start the program

How to setup the database config.ini

The program can be run without a database purely in offline mode, however it will have no collection properties.

To setup the database connection with the program, a “config.ini” file must be created.

Use the following steps to configure the “config.ini” file:

1. Create a new file at services/config.ini

2. Copy and paste the following scaffold:

[Cosmos Connection Parameters]

endpoint = aaaa

database_name = bbbb

key = cccc

3. Locate the URI path from the database Portal “Overview” page

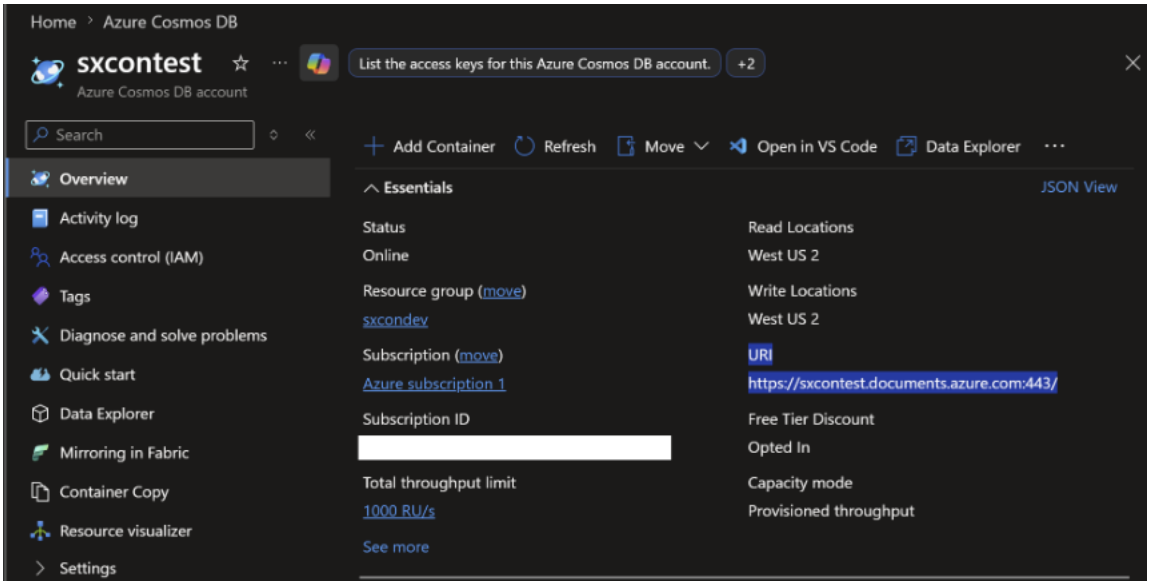

FIGURE 3.1: LOCATING THE URI PATH

4. Copy the URI path from the Portal as found in figure 15.1 and replace “aaaa” as the value for “endpoint” in the config.ini file

5. Locate the database name from “Data Explorer” of the Portal

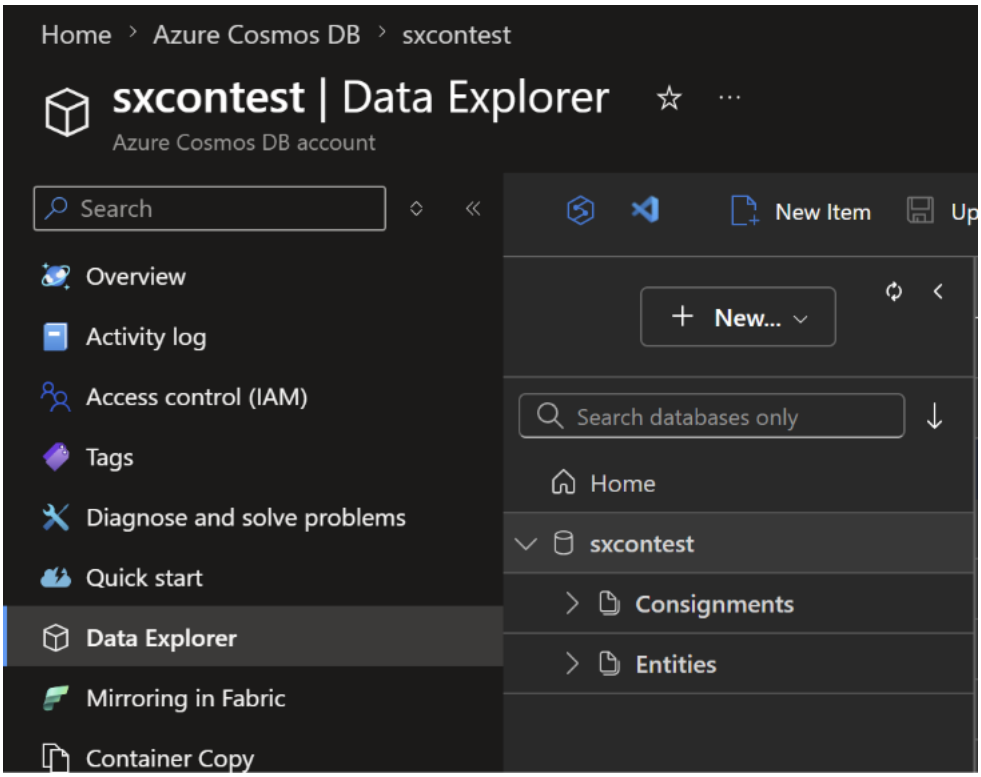

FIGURE 15.2: LOCATING THE DATABASE NAME

6. Copy the database name as found in figure 15.2 and replace “bbbb” as the value for “database_name” in the config.ini file

7. Locate the Primary Key value nested in the “Keys” page from the “Settings” in the left menu bar as shown in figure 15.3

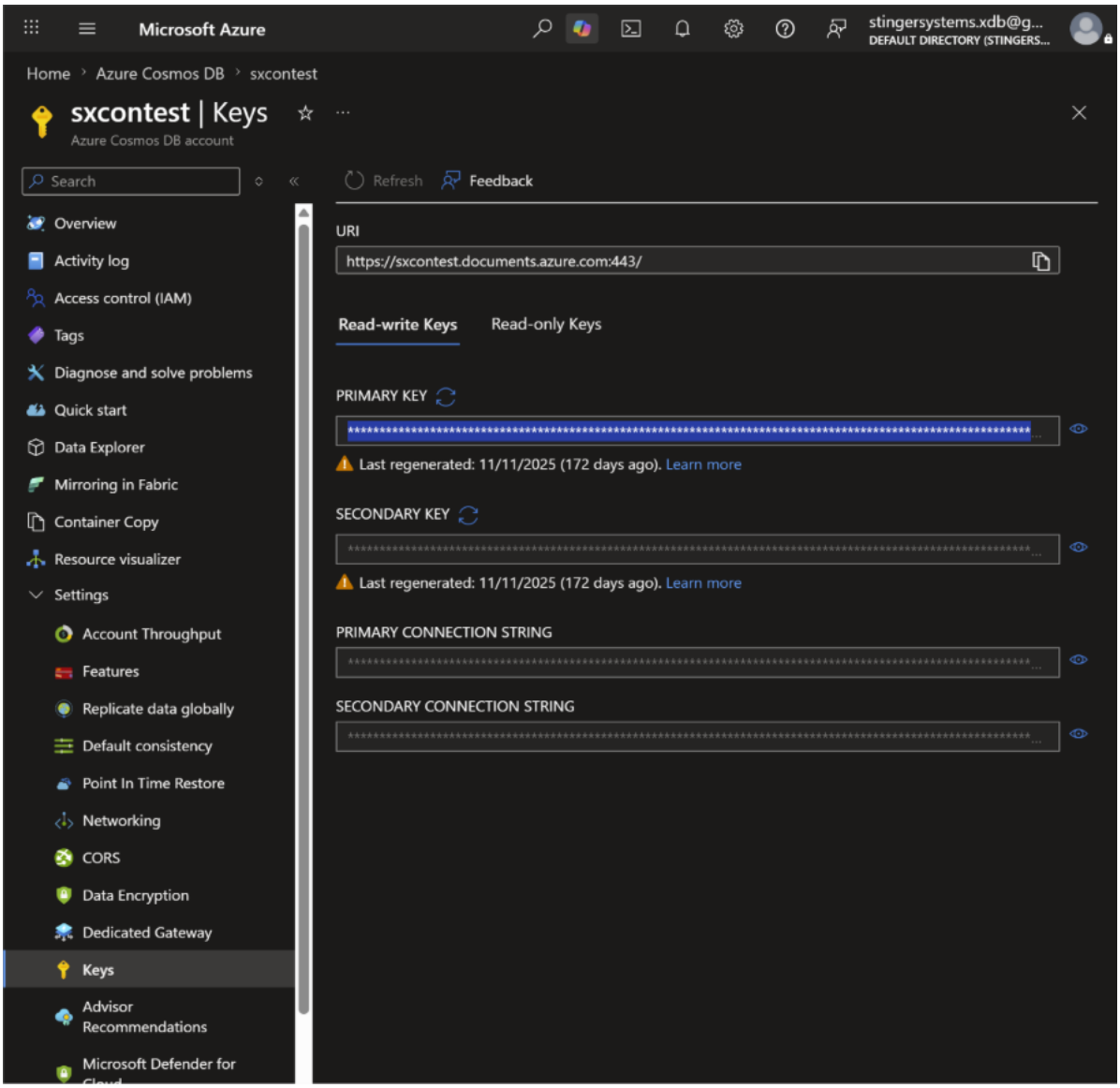

FIGURE 15.3: LOCATING THE PRIMARY KEY VALUE

8. Click on the eye to unhide the Primary Key value

9. Copy the Primary key and replace “cccc” as the “key” value of the config.ini file

10. Save the changes to the config.ini file and restart the program

4. User interactions

A) Login

GUI Aid(s)

FIGURE A.1: THE LOGIN SCREEN

Source Code

Main File:

ui/prompts/login.py

Full path: https://github.com/ifndefy/SX-Con/blob/integration/ui/prompts/login.py

This is the main file for the Login screen. It defines the LoginScreen class, which is a QDialog that displays the login UI and handles all login logic. On initialization, it sets the window title, fixes the width to 400px, sets itself as modal, loads the logo path, builds the UI via setup_ui(), and applies the default theme. The two core methods are:

- attempt_login() — called when the Login button is clicked. Reads the username and password fields, calls authenticate(), and either accepts the dialog (on success) or shows a warning and clears the password field (on failure).
- authenticate() — connects to the Entities container in the database, queries for the username, retrieves the stored password hash, and passes the input password and hash to authenticate_password(). Returns True only if the result is "1". Returns False on blank input, user not found, missing hash, or any exception.

If this file breaks: The login dialog will fail to open or the login button will have no effect. Check that all imports at the top of the file resolve correctly — any missing dependency will prevent the

class from loading. If the login button accepts credentials it should not, the fault is in authenticate(). If the UI layout is wrong, the fault is in setup_ui().

Supporting Files:

src/SPOT.py

Full Path: https://github.com/ifndefy/SX-Con/blob/integration/src/SPOT.py

This is the Single Point of Truth constants file. The login screen uses two values from it:

- SPOT.APP_VERSION — displayed as a revision label (rev_label) centered on the login screen directly below the logo.
- SPOT.OFFLINE — read and written by connect_database.py at startup and on every connection attempt. The login screen does not read it directly, but if the database cannot connect, SPOT.OFFLINE is set to True, which causes db_connection.connect() to return None and makes authenticate() fail for all users.

If this file breaks: If APP_VERSION is missing or renamed, the revision label on the login screen will throw an AttributeError and the screen will fail to build. If OFFLINE is missing, connect_database.py will throw an AttributeError on every connection attempt.

B) Offline Mode

GUI Aid(s)

FIGURE B.1: CREATE NEW TAB WHEN INSTANTIATED IN OFFLINE MODE

Source Code

There is no single file dedicated to offline mode. The program will check src/SPOT.py for Boolean variable “OFFLINE”. If variable “OFFLINE” is True, then the individual checks for online mode embedded in files will fail and the objects will not be instantiated.

It is the intent of the program to immediately attempt to login after logging out via the logout button. Should the program fail to make successful contact with the database, it will immediately log in as offline mode again. Should the program make successful contact with the database then it will instantiate the login screen for seamless transitioning between offline and online mode. During usage in offline mode the program will not attempt to login to not interrupt any ongoing actions should there be any.

Regular User vs Admin Account vs Offline Account

Offline mode bypasses the login process and takes the user straight to the create new tab. No other tabs can be accessed. Tickets cannot be stored in the database in this mode but can still be created, exported, and printed. Consignments created during offline mode can be imported when the program is started in online mode by using the “Sync Offline” button in the “Settings” tab.

C) Forgot Password

GUI Aid(s)

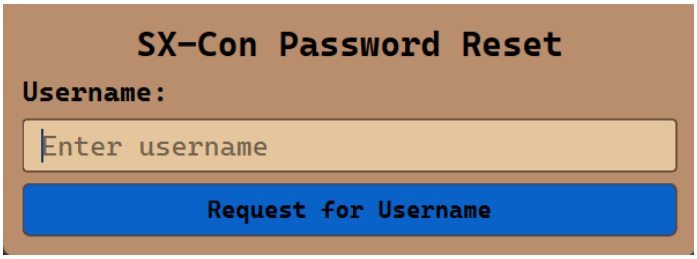

FIGURE C.1: THE INITIAL "FORGOT PASSWORD" POP UP WINDOW

FIGURE C.2: THE SECOND "FORGOT PASSWORD" POP UP WINDOW AFTER A SUCCESSFUL USERNAME INPUT

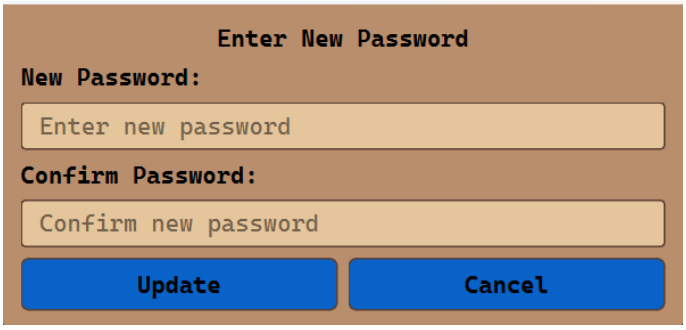

FIGURE C.3: THE FINAL "FORGOT PASSWORD" POP UP WINDOW ENABLING USERS TO ENTER A NEW PASSWORD

Source Code

Main File:

ui/prompts/forgot_pw.py

What it does:

- Sets up windows for entering username and answering security questions, and checks answers against the database.

How does it interact with the Forgot Password prompt:

- Creates the windows, accepts the inputs, and references inputs against database.

What can change:

- Due to the nature of hashing, it is not recommended to change this file

Supporting Files:

ui/prompts/password_dialog.py

What it does:

- Sets up window for setting new password, verifies password is filled and fields match, and adds the new password to the database, overwriting the chosen account's previous password.

How does it interact with the Forgot Password prompt:

- Creates the new password window and sets valid entry into the database

What can change:

- Due to how hashing and confidential data is interweaved, it is not recommended to change this file.

Regular User vs Admin Account vs Offline Account

User and Admin: this feature should function identically for both user and admin level accounts

Offline: this feature is inaccessible in offline mode

D) Create New Tab

GUI Aid(s)

FIGURE D.1: THE CREATE NEW TAB

Source Code

Main File:

ui/tabs/create_new.py

- It is the intent of the program to have much of all data be created through the Create New Tab. This centralizing of data creation makes it easier to have consistent data for all database items. It is not advised to change this file.

Supporting Files:

src/SPOT.py: from src import SPOT

What it does:

- Central resource file that all other files can import from to have the same consistent values

How does it interact with Create New Tab:

- create_new.py imports the variable "APP_VERSION" to display

What can change:

- "APP_VERSION": can be incremented when changes to the source code are made

handlers/handler_print.py: from handlers.handler_print import handler_print

What it does:

- A sequencer to handle printing a physical copy of a consignment ticket
    - Validates that the PDF file exists locally at the path specified from "get_pdf_path"
    - Instantiates an object of sxc_printer, which then executes the printing

How does it interact with Create New Tab:

- Clicking on the print button will call the handler to print a physical copy of the consignment

What can change?

- "base" variable in method "get_pdf_path" of "handler_print.py"
    - should the destination directory of the PDF files be changed, both the "base" variable and return value should be updated accordingly

ui/core/revenue_by_product_type.py: from ui.core.revenue_by_product_type import RevenueByProdType

What it does:

- Calculates revenues by product type using bank currency method to ensure that the fractional products produce a consistent and proportional sum
- Displays the calculated values in a GUI widget

How does it Interact with Create New Tab:

- Calculates the totals by product type and displays them

What can change:

- "product_types" variables within "setup_ui" method can be updated to include additional product types

ui/core/revenue_generation.py: from ui.core.revenue_generation import RevenueGeneration

What it does:

- Calculates revenues by quartiles using the currency banking method to ensure that the fractional products produce a consistent and proportional sum
- Displays the calculated values in a GUI widget

How does it Interact with Create New Tab:

- Calculates revenues and displays them

What can change:

- The calculation method can be changed should it be necessary
    - The calculation is currently as follows and uses bank rounding:
      total: price * quantity * quartile (as a percentage)
      super_x: price * quantity * rate
      vendor: price * quantity * (1 - rate)

utils/core/export_doc.py: from utils.core.export_doc import export_offline_record

What it does:

- Gathers live GUI inputs and stores them into a json file if validation of the values is successful

How does it Interact with Create New Tab:

- While in offline mode, the “Create Record” button is replaced with the “Export Report” button which calls “export_offline_record”

What can change:

- “offline_dir” variable of method "export_offline_record" of file "export_doc.py" to change the output directory of the json files

utils/core/generate_excel.py: from utils.core import generate_excel as xls_gen

What it does:

- Gathers data from the consignment either locally or remotely and generates a xslx file

How does it Interact with Create New Tab:

- Clicking the “Excel” button will trigger the xslx file creation

What can change:

- “filepath” variable in method "generate_excel" of file "generate_excel.py" can be changed to declare a starting location to always store the file or even to autonomously store the file at a static local location

utils/parse_consignment_table.py: from utils.parse_consignment_table import fetch_consignment_data

What it does:

- Pulls data from an external file named “SPOT_CR.csv”

How does it Interact with Create New Tab:

- The pulled data is then passed into the GUI to be displayed and used as the consignment rates for the respective product types

What can change:

- The search path of SPOT_CR.csv can be changed via variable "__csv_location" in method "fetch_consignment_data" in file "parse_consignment_table.py"

validate: import validate as VAL

What it does:

- Validates values using a series of Boolean expressions

How does it Interact with Create New Tab:

- Prior to consignment item creation in the database, a series of Boolean expressions from the "VAL" directory are applied to the gathered data documents from the GUI input fields
    - If the gathered data passes validation, the consignment can be created in the database
        - Else it will abort creation and return to the GUI with a red flash indicating the failure field

What can change:

- Additional Boolean expressions can be added to the individual validation files that can be found within the "VAL" directory to further provide constraints to the possible input values prior to consignment item creation in the database

Regular User vs Admin Account vs Offline Account

In the tab bar, a red tab indicates it is only accessible by Admin level accounts with network access.

In non-admin mode, the red tabs will not instantiate.

In offline mode, only the Create New tab will instantiate as all other tabs require network or admin level access. In Offline mode a dedicated username is logged into the program and does not have admin access to prevent manipulation of data while in offline mode.

Should the user not have admin access, the GUI will instantiate as follows:

FIGURE D.2: THE CREATE NEW TAB WITHOUT ADMIN STATUS

Should the user not have network access when launching the program, the login screen will attempt fail to connect to the database which initiates the program to start in offline mode which does not instantiate any tabs that require database access. The GUI will appear as follows:

FIGURE D.3: THE CREATE NEW TAB WITHOUT NETWORK ACCESS

E) Vendor Tickets Tab

GUI Aid(s)

FIGURE E.1: THE VENDOR TICKETS TAB WITHOUT A QUERY

Source Code

Main File:

ui/tabs/vendor_tickets.py

- The tab itself has minimal source code and relies heavily on imported and fetched records from the database which is then primarily displayed using the “View” import.

Supporting Files:

handlers/handler_print.py: from handlers.handler_print import handler_print

What it does:

- A sequencer to handle printing a physical copy of a consignment ticket
  - Validates that the PDF file exists locally at the path specified from “get_pdf_path”
  - Instantiates an object of sxc_printer, which then executes the printing

How does it interact with Vendor Tickets Tab:

- Clicking on any of the “Print” buttons will call the handler to print a physical copy of the consignment

What can change?

- “base” variable in method “get_pdf_path” of “handler_print.py”
    - should the destination directory of the PDF files be changed, both the “base” variable and return value should be updated accordingly

utils/core/generate_excel.py: from utils.core import generate_excel as xls_gen

What it does:

- Gathers data from the consignment either locally or remotely and generates a xslx file

How does it Interact with Vendor Tickets Tab:

- Clicking on any of the “Excel” button will trigger the xslx file creation

What can change:

- “filepath” variable in method "generate_excel" of file "generate_excel.py" can be changed to declare a starting location to always store the file or even to autonomously store the file at a static local location

Regular User vs Admin Account vs Offline Account

There is no difference between user and admin levels in the Vendor Tickets tab.

In offline mode, the tab will not instantiate as it requires network access.

F) Search Tickets Tab

GUI Aid(s)

FIGURE F.1: THE SEARCH TICKETS TAB WITHOUT A QUERY

Source Code

Main File:

ui/tabs/search_tickets.py

Supporting Files:

handlers/handler_print.py: from handlers.handler_print import handler_print

What it does:

- A sequencer to handle printing a physical copy of a consignment ticket
    - Validates that the PDF file exists locally at the path specified from “get_pdf_path”
    - Instantiates an object of sxc_printer, which then executes the printing

How does it interact with Search Tickets Tab:

- Clicking on any of the “Print” buttons will call the handler to print a physical copy of the consignment

What can change?

- “base” variable in method “get_pdf_path” of “handler_print.py”
    - should the destination directory of the PDF files be changed, both the “base” variable and return value should be updated accordingly

utils/core/generate_excel.py: from utils.core import generate_excel as xls_gen

What it does:

- Gathers data from the consignment either locally or remotely and generates a xslx file

How does it Interact with Search Tickets Tab:

- Clicking on any of the “Excel” button will trigger the xslx file creation

What can change:

- “filepath” variable in method "generate_excel" of file "generate_excel.py" can be changed to declare a starting location to always store the file or even to autonomously store the file at a static local location

Regular User vs Admin Account vs Offline Account

There is no difference between user and admin levels in the Search Tickets tab.

In offline mode, the tab will not instantiate as it requires network access.

G) Settings Tab

GUI Aid(s)

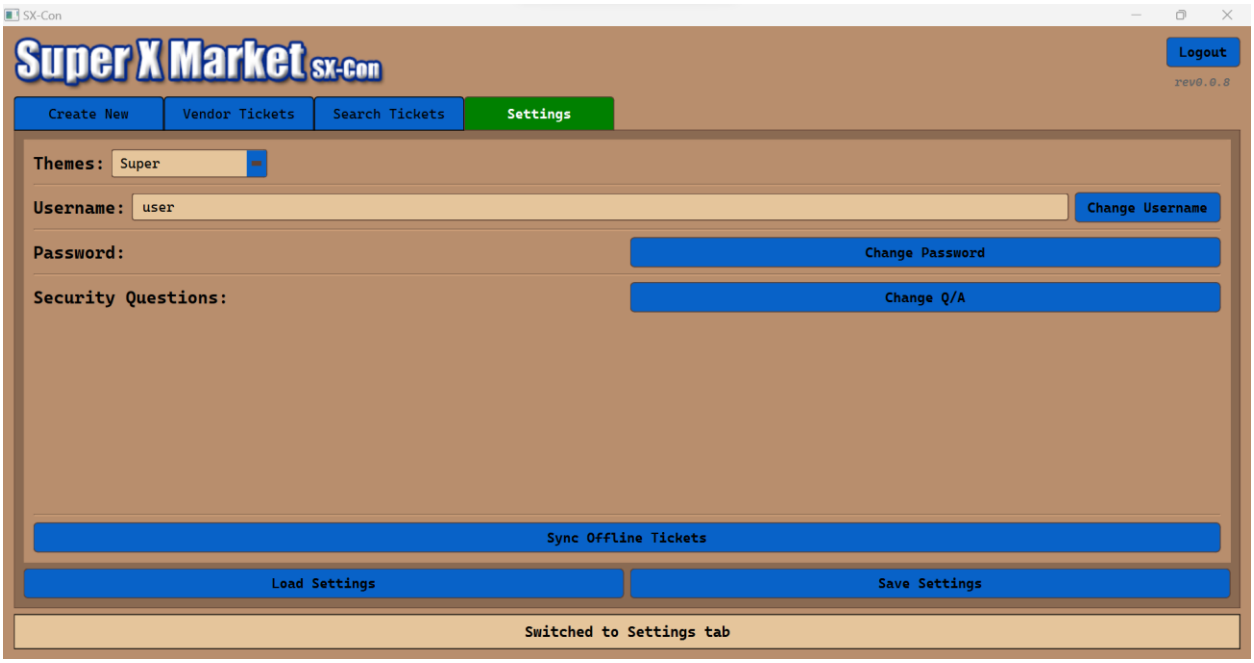

FIGURE G.1: THE SETTINGS TAB

FIGURE G.2: THE SETTINGS TAB WITH THE VERIFY CREDENTIALS WINDOW OPEN (AFTER CLICKING "CHANGE USERNAME", "CHANGE PASSWORD", OR "CHANGE Q/A")

Source Code

Main File:

ui/tabs/settings.py

Supporting Files:

Ui/prompts/login.py

What it does:

- Shows a modal login dialog with username, password, login button, and forgot-password button
- Authenticates credentials against the Entities database container using authenticate_password
- On success, populates src.user.current_user with username and admin status, then accepts the dialog
- On failure, shows a warning and clears/selects the inputs

How it interacts with the Settings tab:

- It passes the theme_manager instance used by the main window and settings tab, so the login UI uses the app theme consistently
- Sets current_user on successful login
- Settings tab reads current_user.get_username() to populate its username field

What can change:

- The entered username and password values
- Authentication result (valid or invalid login)
- Global current_user state (username, admin flag)
- Theme applied to the dialog via theme_manager

ui/prompts/password_dialog.py

What it does:

- Displays a modal dialog for entering a new password and confirming it.
- Validates that the new password is non-empty and that both fields match.
- If valid, stores the value in self.new_password and accepts the dialog.

How it interacts with the Settings tab:

- SettingsTab.show_password_change_dialog() opens this dialog after credential verification.
- On accept, SettingsTab.perform_password_update(dialog.new_password) updates the password in the database and logs the user out.

What can change:

- new_password_input text
- confirm_password_input text
- self.new_password once the dialog is accepted

ui/prompts/security_dialog.py

What it does:

- Displays a modal dialog for selecting two security questions and entering two answers.
- Validates that both questions are selected and both answers are non-empty.
- Hashes the question/answer pairs using hash_security_question_answer.
- Stores the hashed result in self.hashed_questions_answers and accepts the dialog.

How it interacts with the Settings tab:

- SettingsTab.show_qa_change_dialog() opens this dialog after credential verification.
- On accept, SettingsTab.perform_qa_update(dialog.hashed_questions_answers) writes the hashed security Q/A to the database.

What can change:

- selected question 1 and question 2
- answer 1 and answer 2 text
- self.hashed_questions_answers after hashing succeeds

Critical Code:

src/core/authenticate.py

What it does:

- The main authentication function that:
  - ensures both inputs are strings,
  - encodes the plain password and stored hash as UTF-8 bytes,
  - uses bcrypt.checkpw(password_bytes, hash_bytes) to verify the password against the stored bcrypt hash,
  - returns "1" on success and "-1" on failure.

src/core/hash_password.py

What it does:

- Generates bcrypt hashes for passwords.
- Used whenever a password is created or updated.

src/core/hash_qa.py

What it does:

- Hashes security questions and answers using the same password hashing logic.
- Used by security question update and forgot-password flows.

Recommendations:

- Because these source code are core to password validation and login security, it is not recommended to change authenticate.py unless you fully understand bcrypt and authentication impacts.

Regular User vs Admin Account vs Offline Account

There is no difference between user and admin levels in the Settings tab.

In offline mode, the tab will not instantiate as it requires network access.

H) Admin Settings Tab

GUI Aid(s)

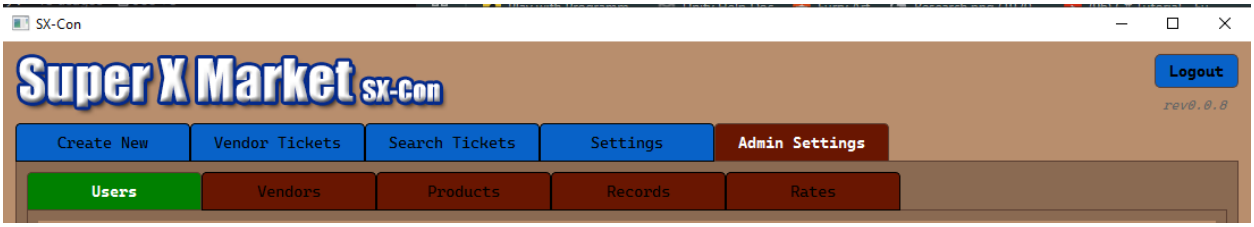

FIGURE H.1: WHAT SUBTABS FOR ADMIN SETTINGS LOOK LIKE

Source Code

Main file

ui/tabs/admin_settings.py

This is the main file for the admin settings screen. This is where subtabs are setup and initiated. When the application is offline, it would only initiate subtabs that are ok to be initiated in admin settings. On initialization, it would setup its database connection. There are two methods that comes with the admin_settings.py:

- setup_ui(self): — The function initializes the main layout for other subtabs to be put in.
- setup_subtabs(self): — The function initiates all the subtab and links them up with the subtab QTabWidget(). Each new subtab class that is initiated, it would be given the parameter self.apihandler and/or self.db_connection. It does not mean when you create a new tab you must incorporate self.apihandler or self.db_connection.
- on_tab_changed(): — It logs into the console about a change in tab

If this file breaks: The admin settings won't appear at all. Check if you have all imports at the top of the file, as if you don't have all dependencies, it will prevent this class from loading. If a certain tab is not showing up check if you have used self.tabs.addTab() and incorporate the tab.

Regular User vs Admin Account vs Offline Account

As the namesake, the Admin Settings tab is only accessible and instantiated when the user is logged in with admin privileges.

In offline mode, the tab will not instantiate as it requires network access.

I) Users Subtab

GUI Aid(s)

FIGURE I.1: WHAT ADMIN SETTING USER SUBTAB WITHOUT A QUERY LOOKS LIKE

FIGURE I.2: WHAT ADMIN SETTING USER SUBTAB WITH A QUERY LOOKS LIKE

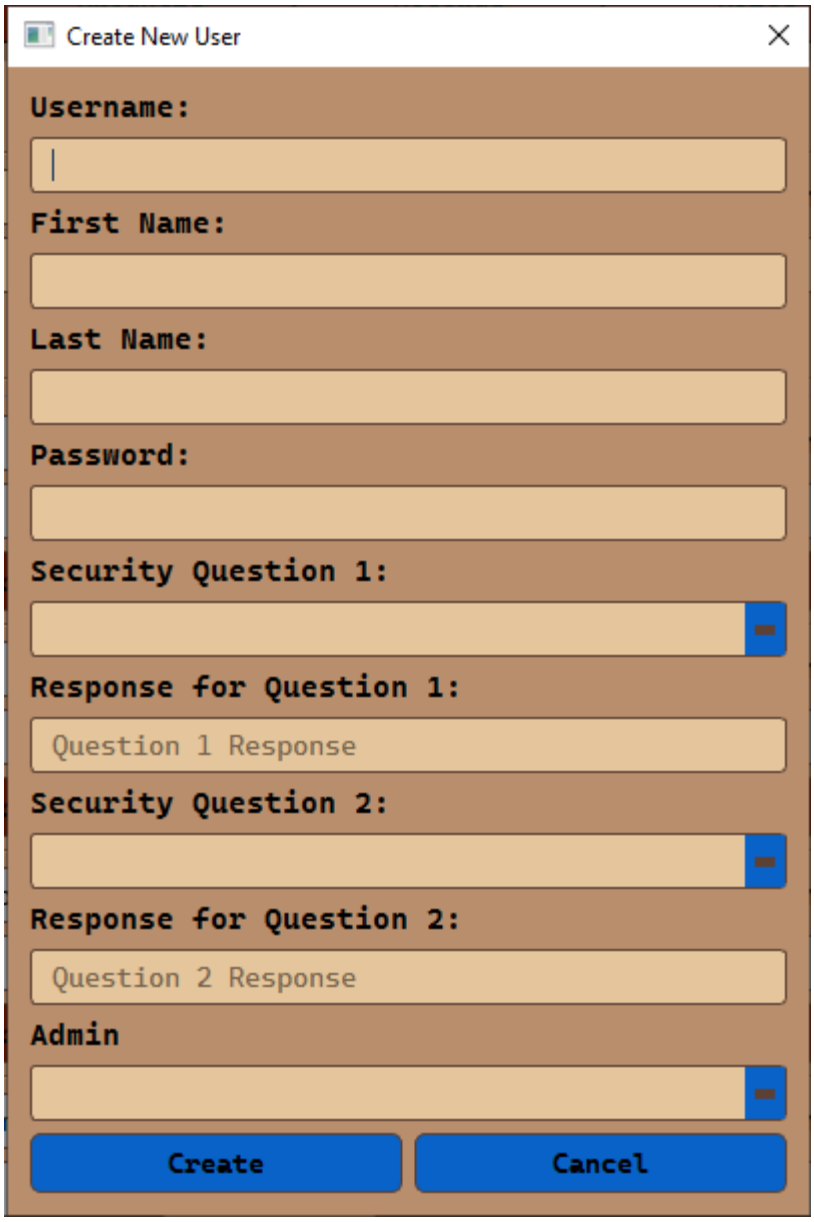

FIGURE I.3: WHAT CREATE NEW USER WINDOW LOOKS LIKE

Source Code

Main File

ui/tabs/subtabs/users.py

This is the main file for the user screen as a subtab in admin settings. Its purpose is to allow the user to view all or a selection of users and allow the user to edit them.

The UI is setup in the setup_ui() function; it first creates the box layout then creates the header, which contains the title and a “Create New User” button. The second thing the UI does is to create search section rows, where each search section row holds its own texts, or buttons. Here is a list of problems that you might encounter and potential fixes/look at for

- If certain buttons, or text won't show up, check the item in setup_ui() to see if they are connected to the section they belong to.
- If the whole row of section does not show, then check if the whole row is linked up with the main layout
- If nothing shows up then check if there are any PyQt6.QtWidgets imports missing, as the import is used for creating all the widgets, tabs, text, and buttons.

The function for linking all button handling functions together from the setup_ui() is setup_button_connections(). The functions that setup_button_connections link up with are

- Clear() — clears the search history in the main layout (Figure I.1)
- Build_and_search() — It takes in all the inputs and does a query. Each returned data is then put into the search history (Figure I.2)
- Create_new_user_prompt() — it will open a new window that will prompt the user on creating a new user account (Figure I.3)

The function for the layout of a user section is on add_user_section(). It will build a user section with the given input it asks for. The function will also add two new buttons: the edit button and the Reset password.

- Edit button will allow the admin to change the user Username, First name, Last name and admin privileges
- Reset password will allow the admin to change the user's password

The function query_db interacts with the database. It will grab all relavent records based on the inputs given.

Supporting Files

- Ui/tabs/base.py — the class that is inherited from
- Ui/prompts/password_dialog.py — when reset password button is called it would call this class
- Src/SPOT.py — uses only the QUESTION_LIST from here
- Services —it is a folder with full database interaction functions.

J) Vendors Subtab

GUI Aid(s)

FIGURE J.1: WHAT THE ADMIN SETTING VENDOR SUBTAB WITH NO QUERY LOOKS LIKE

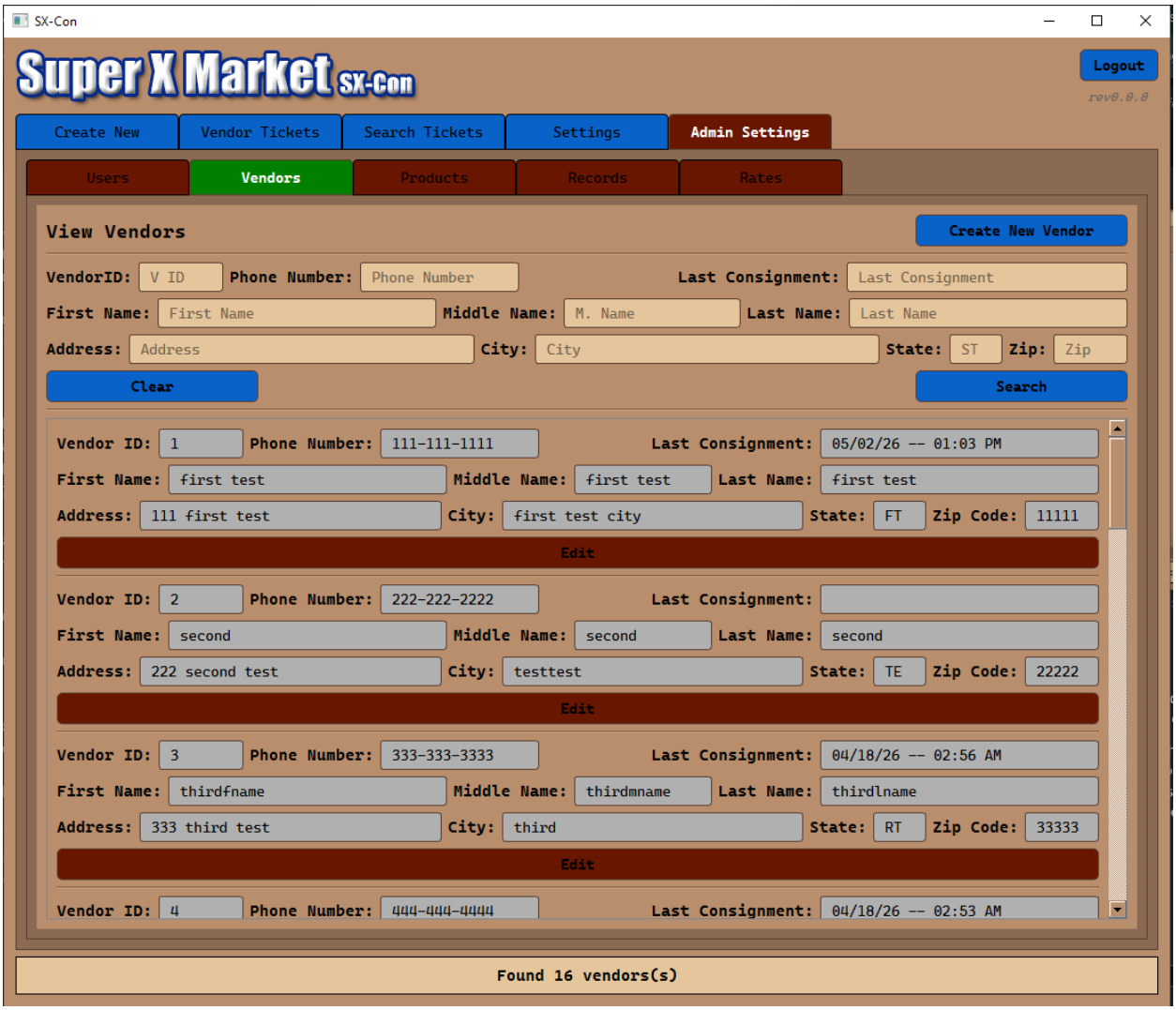

FIGURE J.2: WHAT THE ADMIN SETTING VENDOR SUBTAB WITH A QUERY LOOKS LIKE

FIGURE J.3: WHAT CREATE NEW VENDOR WINDOW LOOKS LIKE

Source Code

Main File

ui/tabs/subtabs/vendors.py

This is the main file for the vendor screen as a subtab in admin settings. Its purpose is to allow the user to view all or a selection of vendors and allow the user to edit them.

The UI is setup in the setup_ui() function; it first creates the box layout then creates the header, which contains the title and a “Create New Vendor” button. The second thing the UI does is to create search section rows, where each search section row holds its own texts, or buttons. Here is a list of problems that you might encounter and potential fixes/look at for

- If certain buttons, or text won't show up, check the item in setup_ui() to see if they are connected to the section they belong to.
- If the whole row of section does not show, then check if the whole row is linked up with the main layout
- If nothing shows up then check if there are any PyQt6.QtWidgets imports missing, as the import is used for creating all the widgets, tabs, text, and buttons.

The function for linking all button handling functions together from the setup_ui() is setup_button_connections(). The functions that setup_button_connections link up with are

- Clear() — clears the search history in the main layout (Figure J.1)
- Build_and_search() — It takes in all the inputs and does a query. Each returned data is then put into the search history (Figure J.2)
- Create_new_vendor_prompt() — it will open a new window that will prompt the user on creating a new vendor (Figure J.3)

The function for the layout of a vendor section is on add_vendor_section(). It will build a vendor section with the given input it asks for. The function will also add one new button: the edit button

- Edit button will allow the admin to change the vendor's Phone number, First name, Middle name, Last name, address, city, state, and zip code

The function query_db interacts with the database. It will grab all relavent records based on the inputs given.

Supporting Files

- Ui/tabs/base.py — the class that is inherited from
- Services —it is a folder with full database interaction functions.

K) Products Subtab

GUI Aid(s)

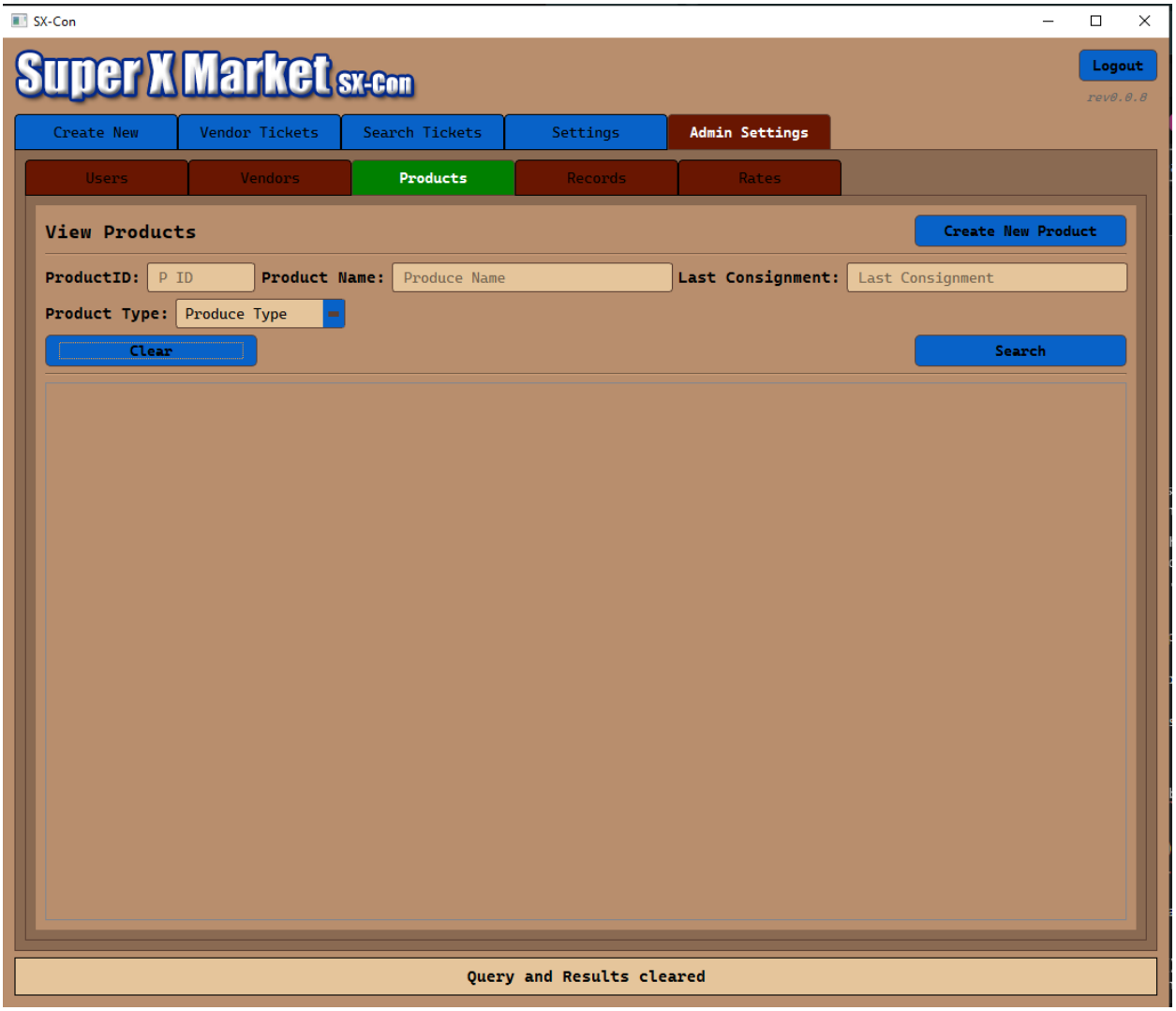

FIGURE K.1: WHAT ADMIN SETTING USER SUBTAB WITH NO QUERY LOOKS LIKE

FIGURE K.2: WHAT ADMIN SETTING USER SUBTAB WITH A QUERY LOOKS LIKE

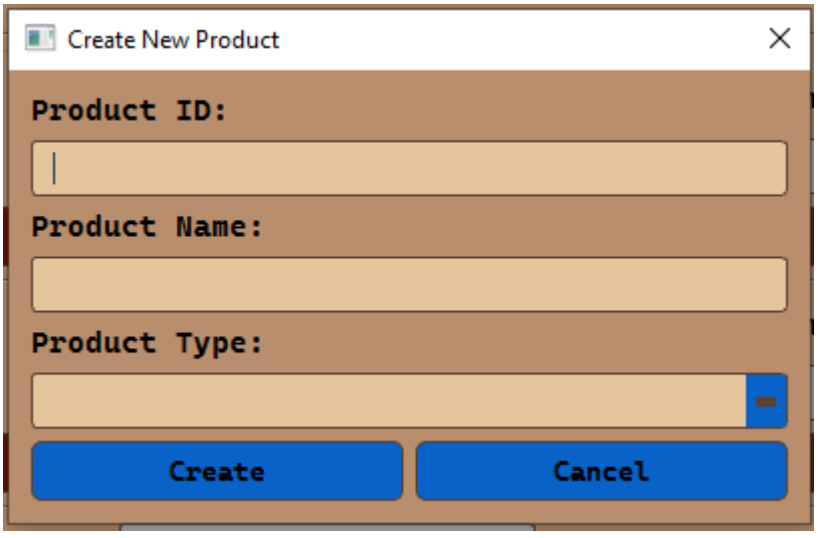

FIGURE K.2: WHAT CREATE NEW PRODUCT WINDOW LOOKS LIKE

Source Code

Main File:

Ui/tabs/subtabs/products.py

This is the main file for the products screen as a subtab in admin settings. Its purpose is to allow the user to view all or a selection of products and allow the products to edit them.

The UI is setup in the setup_ui() function; it first creates the box layout then creates the header, which contains the title and a “Create products User” button. The second thing the UI does is to create search section rows, where each search section row holds its own texts, or buttons. Here is a list of problems that you might encounter and potential fixes/look at for

- If certain buttons, or text won’t show up, check the item in setup_ui() to see if they are connected to the section they belong to.
- If the whole row of section does not show, then check if the whole row is linked up with the main layout
- If nothing shows up then check if there are any PyQt6.QtWidgets imports missing, as the import is used for creating all the widgets, tabs, text, and buttons.

The function for linking all button handling functions together from the setup_ui() is setup_button_connections(). The functions that setup_button_connections link up with are

- Clear() — clears the search history in the main layout (Figure K.1)
- Build_and_search() — It takes in all the inputs and does a query. Each returned data is then put into the search history (Figure K.2)
- Create_new_product_prompt() — it will open a new window that will prompt the user on creating a new product (Figure K.3)

The function for the layout of a user section is on add_ product_section(). It will build a user section with the given input it asks for. The function will also add two new buttons: the edit button.

- Edit button will allow the admin to change the product name and type.

The function query_db interacts with the database. It will grab all relavent records based on the inputs given.

Supporting Files

- Ui/tabs/base.py — the class that is inherited from
- Services —it is a folder with full database interaction functions.

L) Records Subtab

GUI Aid(s)

FIGURE L.1: THE RECORDS SUBTAB WITHOUT A QUERY

Source Code

Main File:

ui/tabs/subtabs/records.py

Supporting Files:

handlers/handler_print.py: from handlers.handler_print import handler_print

What it does:

- A sequencer to handle printing a physical copy of a consignment ticket
    - Validates that the PDF file exists locally at the path specified from “get_pdf_path”
    - Instantiates an object of sxc_printer, which then executes the printing

How does it interact with Records Subtab:

- Clicking on any of the “Print” buttons will call the handler to print a physical copy of the consignment

What can change?

- “base” variable in method “get_pdf_path” of “handler_print.py”
    - should the destination directory of the PDF files be changed, both the “base” variable and return value should be updated accordingly

utils/core/generate_excel.py: from utils.core import generate_excel as xls_gen

What it does:

- Gathers data from the consignment either locally or remotely and generates a xslx file

How does it Interact with Records Subtab:

- Clicking on any of the “Excel” button will trigger the xslx file creation

What can change:

- “filepath” variable in method "generate_excel" of file "generate_excel.py" can be changed to declare a starting location to always store the file or even to autonomously store the file at a static local location

M) Rates Subtab

GUI Aid(s)

FIGURE M.1: THE RATES SUBTAB (ADMIN VIEW)

FIGURE M.2: THE RATES SUBTAB (OFFLINE VIEW)

FIGURE M.3: THE RATES SUBTAB (USER VIEW)

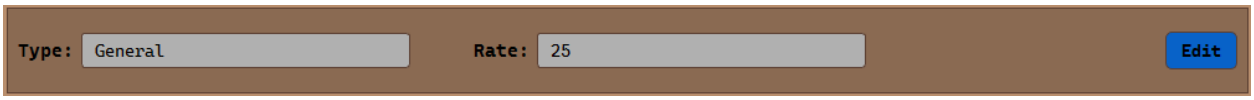

FIGURE M.4: A LOCKED PRODUCT TYPE 'TICKET'

FIGURE M.5: AN UNLOCKED 'TICKET'

Source Code

Main File

ui/tabs/subtabs/rates.py - Contains the actual logic for the tab's layout and behavior. Split into two objects: CRTab, which handles the high-level layout of the tab and setting up the containers for everything within the subpage and __CREntry, which handles the individual entries in our table to properly display the rows of the corresponding table and handles linking buttons to limit them to the scope of each individual entry box.

Also contains a private non-instanced helper method __write_to_spot_csv, which handles the actual operation to write out to the program's SPOT table, so no other objects can replicate the behavior by importing the same modules.

Supporting Files

ui/tabs/base.py - This contains the program wide template/interface that all tabs must follow, contains some helper scaffolding to manage switching between tabs in a consistent manner.

utils/logger.py - Contains a handle to a global logger instance and exposes a set of functions to easily error messages and warnings while maintaining a consistent log format.

utils/parse_consignment_table.py - Contains a helper function for reading the program's consignment rate table and parsing it into a consistent format for other objects to reference.

Regular User vs Admin Account vs Offline Account

Regular User or Offline User: The user should expect to not be able to access or see the rates subtab, since it is a child of the 'Admin Settings' panel (see fig. M.2, M.3).

Admin User: Once in admin mode, the user can access the rates section in the *'Admin Settings'* section. This section should automatically populate with the contents of the programs SPOT_CR.csv (see fig. M.1).

The rates information for the program is meant to assist in record creation and as such only exists locally for each instance of the program, editing an entry in the subsection will only update your numbers for future use and will not be reflected in the tables of other instances of the program on other computers.

Each section in the subtab represents a row in the SPOT_CR.csv file with two input fields of the same name. For each row in SPOT_CR.csv, a section will be created in Rates subtab. The "Type" field in both the subtab and csv file should be a unique product type. The "Rate" field in both the subtab and csv file should be the associated consignment rate for said product type. In order to change the rate value, the section must first be unlocked via the "Edit" button. It is not advised to add a product type into the csv file as the source code is hardcoded in many places to only the initial 3 of "Hot Food", "General", and "Produce".

Once unlocked, a product's rate can be edited freely. The field will automatically reject non-number keystrokes. The edit button is also replaced with two new buttons for the corresponding entry box, *'Cancel'* and *'Save'* (see fig. 3.1).

Pressing *'Cancel'* will swap the field back to the value it originally had when editing was first unlocked and reset the state, re-locking the fields (see fig. 2.1).

Choosing *'Save'* will run a final check on the field to ensure that the corresponding entry is valid and write to the SPOT file before re-locking the fields. If the data is invalid: then the callback function is aborted, nothing is written, and the state returns to what it was before 'Save' was pressed (see fig. 2).

5. Database

Database information

Provider: Microsoft Azure

Database Type: Cosmos NoSQL

Service Subscription: Free tier (instant rebate)

Service Type: Platform as a Service

The provider will handle all server-side operations

Login Address: https://portal.azure.com/auth/login/

ERD

Due to the nature of a NoSQL database there is no direct schema between the containers. Along with the limitation of the free tier which sets the maximum number of containers to 2, the items have been divided into either a Consignment or an Entity. Where any user, vendor, or product is filed as an entity.

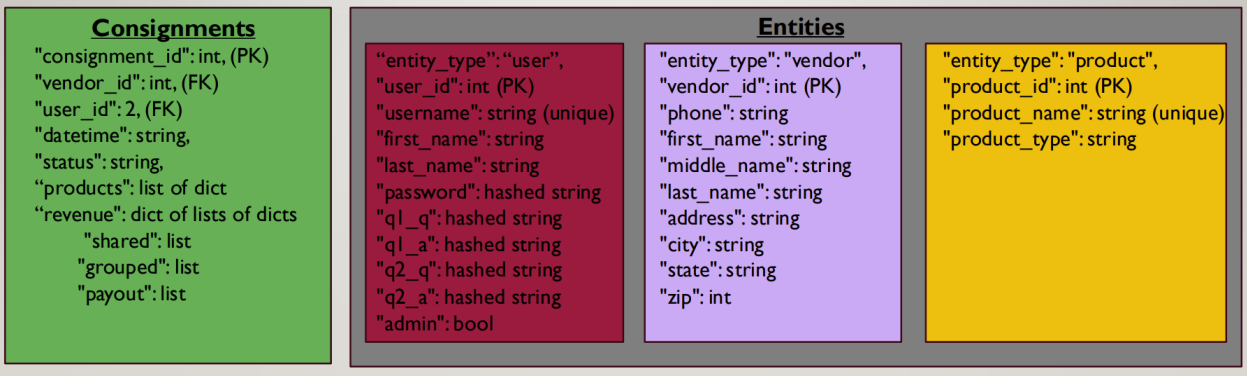

FIGURE 5.1: DATABASE DESIGN WITH 2 CONTAINER LIMITATION

Program Dataflow in Relation with the Database

It is the intent of the program to have the near all data be created within the “Create New Tab”. Supporting data can be created in the “View”, “Payout”, and Admin subtabs.

FIGURE 5.2: THE POPULATED CREATE NEW TAB

"View"

The "View" GUI is nested in a button with the same name within the results of Consignment queries. Clicking on the "View" button will expand the query result's GUI and display the consignment data. While the immediate Consignment data cannot be manipulated to preserve data integrity, the quantities of sold products can be updated to calculate a payout.

FIGURE 5.3: THE EXPANDED "VIEW" BUTTON

The View GUI is imported to the calling tabs through the file ui/core/view_ticket.py. Meaning changing the file will also change all of its imported locations. It is through the View GUI that additional data can be appended or changed for the Consignment data. The View GUI itself is only capable of changing the property "qty_sold" quantity. Through the nested "Payout" widget, payout data can be appended to the Consignment item. A payout can only be created if the qty_sold value has been increased. Decreasing the value will not trigger a payout and will require direct database manipulation of the consignment data to revert the change.

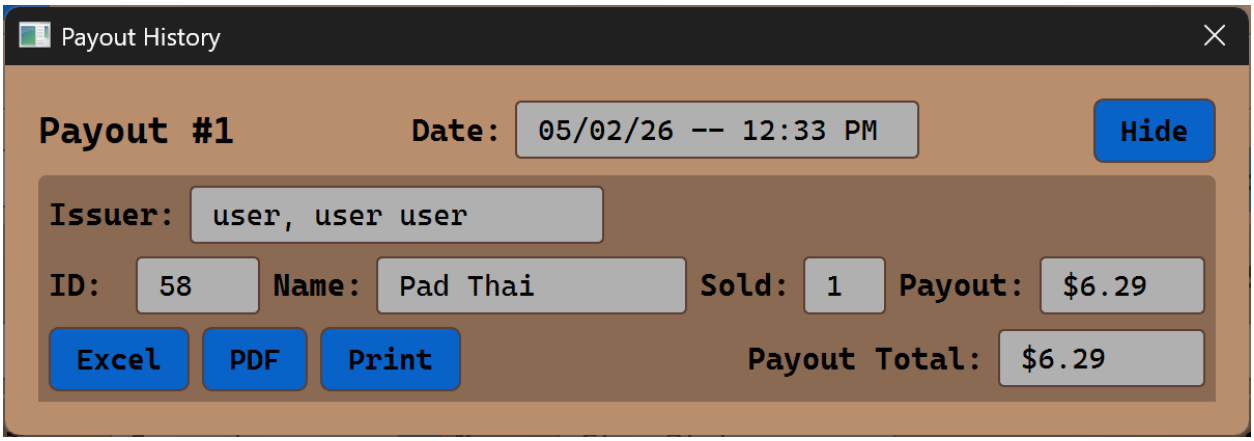

FIGURE 5.4: THE PAYOUT WINDOW

It is also through the View GUI that a ticket can have its status changed to either “OPEN” or “CLOSED”. This action can be done with a button, green or red respectively, by the same name. Only one of the two buttons will appear depending on the current state of the consignment item. If a consignment item is in closed state, it cannot have property changes. Only consignments in open state can have quantity changes or payouts.

Database Issues

How do they restart/recover?

The provider, Microsoft Azure, will have the database in a ready state at all times. This was one of the reasons to select this provider. There is no option for the user to restart the server, in order to do so the support team must be contacted.

How to restore from backup

The provider Microsoft Azure will handle backups autonomously, however restoring a backup is done by the user.

To restore a backup:

1. Log into the Azure portal

2. Menu Search bar, enter the following:

Point In Time Restore

3. Select the desired options

4. Select Submit

How to reactivate the database if it becomes idle

The database should never go into idle mode. If it does, the provider’s support team should be contacted.

What to do if the database is corrupted?

Should the database become corrupted, it is advised to restore the database to a working point using the steps above.